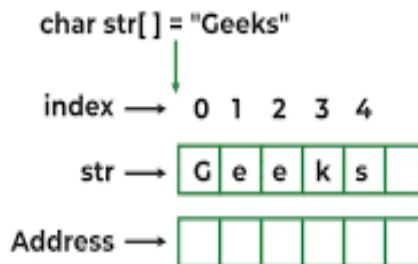


➤ String in C :-

What is a String in C ?

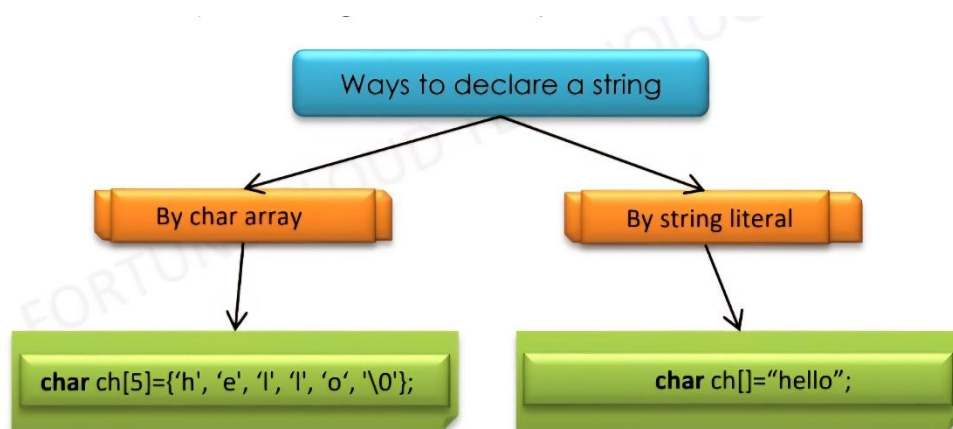
- A string is defined as a sequence of characters terminated by a null character ('\0').
- It is represented using a character array.

String in C



Two ways to declare a String :-

In C programming, strings are declared using two ways character arrays & String Literal.



1. By Character Array

Theory :-

1. Declares each character individually.
2. You must include the '\0' (null character) manually.
3. Useful for character-by-character control.
4. The size must account for the null terminator.

Syntax :-

```
char ch[6] = { 'h', 'e', 'l', 'l', 'o', '\0' };
```

Example :-

```
#include <stdio.h>
```

```
int main()
{
    char ch[6] = { 'h', 'e', 'l', 'l', 'o', '\0' };
    printf("%s", ch);
    return 0;
}
```

2. By String Literal**Theory :-**

1. Easiest and most common way to declare a string.
2. The null terminator '\0' is added automatically.
3. More readable and cleaner code.
4. Size is auto-calculated if not specified.

Syntax :-

```
char ch[] = "hello";
```

Example :-

```
#include <stdio.h>
```

```
int main()
{
    char ch[] = "hello";
    printf("%s", ch);
    return 0;
}
```

➤ What is a Null Character in C?

Definition :-

The null character in C is a special character '\0' (ASCII value 0) used to terminate a string.

- It marks the end of a string.
- Without it, C functions (like printf, strlen) wouldn't know where the string ends.
- It is automatically added when you initialize a string using a string literal.

Example:

```
char str[] = "Hello";
```

➤ gets() Function in C :-

Theory :-

- gets() is used to read a string (including spaces) from standard input.
- It continues reading until a newline (\n) is encountered.

Syntax :-

```
char str[50];  
gets(str);
```

➤ puts() Function in C :-

Theory :-

- puts() is used to print a string to the output.
- It automatically adds a newline at the end.

Syntax :-

```
char str[50];  
puts(str);
```

➤ String Function :-

Before using this string function we should mention program in <string.h> header file .

1.strlen(str)- Returns the length of the string (without "\0") .

```
strlen("Hello"); //Output: 5
```

2.strcpy(dest, src)- Copies one string to another.

```
strcpy(a, "Hi");
```

3.strcat(dest, src)- Concatenates (joins) two strings .

```
strcat(a, " World");
```

4.strcmp(str1, str2)- Compares two strings.

- Returns 0 if equal, positive or negative value otherwise

```
Strcmp("Hi", "HI");
```

Output:0

5. strrev(str)- Reverse a String

- Reverses the characters of a string.

```
char str[50] = "Hello"; strrev(str);
```

Output: "olleH"

6. strlwr(str)- Convert to Lowercase

- Converts all letters in the string to lowercase.

```
char str[50] = "HELLO";
```

```
strlwr(str);
```

Output: "hello"

7.strupr(str)- Convert to Uppercase

- Converts all letters in the string to uppercase

```
char str[50]= "hello";
```

```
strupr(str);
```

Output: "HELLO"

8. strstr(str1, str2)- Find Substring

- Searches for first occurrence of string str2 inside str1.
- Returns a pointer to the first match (or NULL if not found)

```
char str1[] = "Hello World";
```

```
char *result = strstr(str1, "World");
```

Output: World